

AI Student Routine Manager

Complete JIRA Project Setup Guide

For Teammates - Project Management Course (Spring 2025-2026)

Introduction

This document is a readable version of the JIRA guide content from your docx script. Use it as the source-of-truth checklist to create and organize the project in JIRA.

Project overview:

- **Name:** AI Student Routine Manager
 - **Type:** Full-stack web application
 - **Purpose:** Students input class schedules and habits; AI generates realistic gap-free daily routines.
 - **Stack:** Next.js, Tailwind, Supabase, Claude (primary), Gemini/OpenRouter/Groq fallbacks, Vercel, Telegram Bot API
 - **Constraint:** Free tiers only
-

Section 1 - Project Type: Scrum

Select **Scrum** (not Kanban) because the project has:

- A fixed semester deadline (14 weeks)
- Clear scope and defined backlog
- Time-boxed delivery in sprints
- Formal roles and review cycles

Scrum points to mention in report:

- Fixed-length sprints
 - Prioritized product backlog
 - Sprint planning, review, retrospective
 - Acceptance criteria per story
 - Story point estimation (Fibonacci)
-

Section 2 - JIRA Setup Steps

1. Create JIRA account at jira.atlassian.com
 2. Create project:
 - Template: **Scrum**
 - Name: **AI Student Routine Manager**
 - Key: **ASRM**
 - Type: **Team-managed**
 3. Assign roles:
 - Product Owner
 - Scrum Master
 - Developers
 4. Create sprints:
 - Sprint 1: Foundation (Weeks 1-3)
 - Sprint 2: AI Core (Weeks 4-7)
 - Sprint 3: Dashboard & Alerts (Weeks 8-11)
 - Sprint 4: Deployment & Polish (Weeks 12-14)
-

Section 3 - Epics

Create these 6 epics:

5. **ASRM-E1** User Authentication & Account Management
 6. **ASRM-E2** 9-Screen Onboarding Questionnaire
 7. **ASRM-E3** AI Schedule Generation Engine
 8. **ASRM-E4** Interactive Dashboard & Timeline
 9. **ASRM-E5** Telegram Notification System
 10. **ASRM-E6** Deployment, Performance & Production Setup
-

Section 4 - Stories (ASRM-1 to ASRM-20)

Story groups:

- **Authentication:** ASRM-1, ASRM-2, ASRM-3
- **Onboarding:** ASRM-4, ASRM-5, ASRM-6, ASRM-7
- **AI Engine:** ASRM-8, ASRM-9, ASRM-10, ASRM-11
- **Dashboard:** ASRM-12, ASRM-13, ASRM-14, ASRM-15

- **Telegram:** ASRM-16, ASRM-17
- **Deployment:** ASRM-18, ASRM-19, ASRM-20

Each story should include:

- Summary
 - User story statement
 - Acceptance criteria
 - Priority
 - Story points
 - Epic link
 - Sprint
-

Section 5 - Tasks

Tasks are linked technical actions under stories.

Examples in your original script include detailed task sets under:

- ASRM-1 (Sign-Up)
 - ASRM-2 (Sign-In)
 - ASRM-3 (Route Protection)
 - ASRM-8 (Pre-computation)
 - ASRM-9 (AI Fallback Chain)
 - ASRM-12 (Timeline UI)
 - ASRM-17 (Telegram Alerts)
-

Section 6 - Bugs

Create and document these bugs:

- **ASRM-B1:** Gemini model deprecation issue
- **ASRM-B2:** Dashboard rendered unstyled
- **ASRM-B3:** Runtime Error: [object Event]
- **ASRM-B4:** Illogical AI schedule generation
- **ASRM-B5:** Windows npm execution policy issue
- **ASRM-B6:** Onboarding data shape mismatch

Each bug should include:

- Steps to reproduce
- Expected result

- Actual result
 - Root cause
 - Fix applied
 - Files changed
 - Priority, points, status, sprint
-

Section 7 - Sprint Summary

- **Sprint 1:** 21 points
 - **Sprint 2:** 39 points
 - **Sprint 3:** 26 points
 - **Sprint 4:** 10 points
 - **Grand total:** 96 points
-

Section 8 - Teammate Checklist

11. Setup account and project
 12. Create 4 sprints
 13. Create 6 epics
 14. Create all stories
 15. Create all tasks
 16. Create all bugs
 17. Start Sprint 1
 18. Capture required screenshots for report appendix
-

Section 9 - Glossary

Include definitions for:

- Epic
- Story
- Task
- Bug
- Subtask
- Sprint

- Backlog
 - Story Points
 - Acceptance Criteria
 - Product Owner
 - Scrum Master
 - Sprint Planning
 - Sprint Review
 - Burndown Chart
 - Epic Link
-

Optional: Generate .docx from your original script

You already have the full docx builder code. To generate a Word file directly in this repo:

19. Save your JS as `scripts/generate-jira-guide.js`
20. Ensure dependency is installed:
 - `npm install docx`
21. Update output path in script to:
 - `./JIRA_Complete_Guide.docx`
22. Run:
 - `node scripts/generate-jira-guide.js`